

Symbolic Execution: Theory

1

How do *you* test?

- Who has had a bug in their code they *didn't* expect?
- Has anyone had code they were *sure* was correct?
 - How did you know? How would you know?
 - Did you exhaustively test?
 - Is exhaustive testing even practical?

What if exhaustive testing was possible?

2

Outline

- **Concrete Execution**
- **Symbolic Execution Theory**
 - How to execute symbolically
 - Min function example
 - Negative function exercise
 - Limitations
- Symbolic Execution Tools
 - Binary Search function example
 - Discussion

3

Symbolic vs Concrete Execution

- **Concrete, or standard, execution** uses *concrete* values for variables (e.g., 5 for an integer) that represent a single discrete value.
- **Symbolic execution** uses *symbolic* values for variables (e.g., the symbol *a* for an integer) that represent a range of values.

4

Concrete Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if(B<Z)
    Z = B;
  if(Z>A || Z>B)
    assert false;
  return Z;
}
```

A=1, B=2

5

Concrete Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if(B<Z)
    Z = B;
  if(Z>A || Z>B)
    assert false;
  return Z;
}
```

A=1, B=2

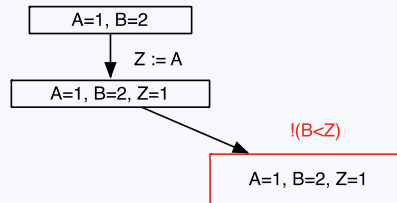
Z := A

A=1, B=2, Z=1

6

Concrete Execution: Example

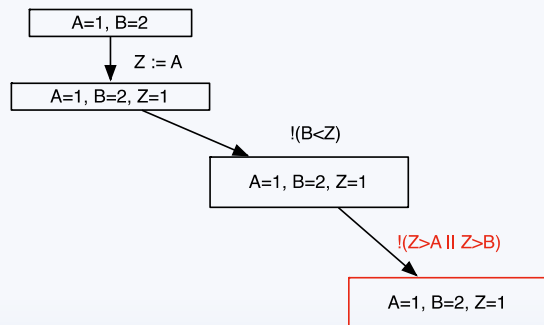
```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```



7

Concrete Execution: Example

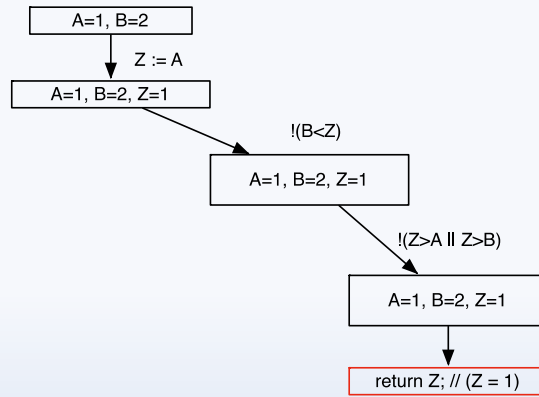
```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```



8

Concrete Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if(B<Z)
    Z = B;
  if(Z>A || Z>B)
    assert false;
  return Z;
}
```



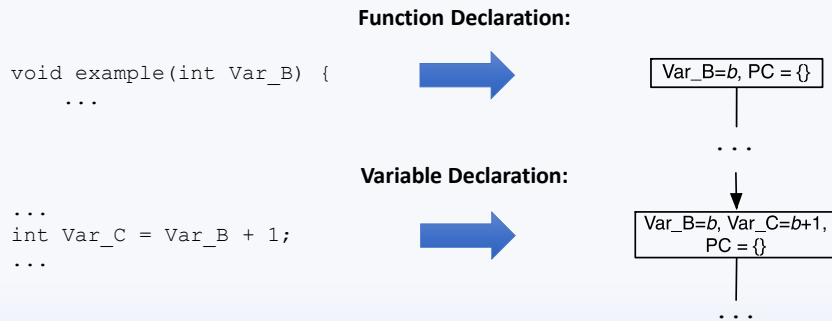
9

Symbolic Execution: Variables



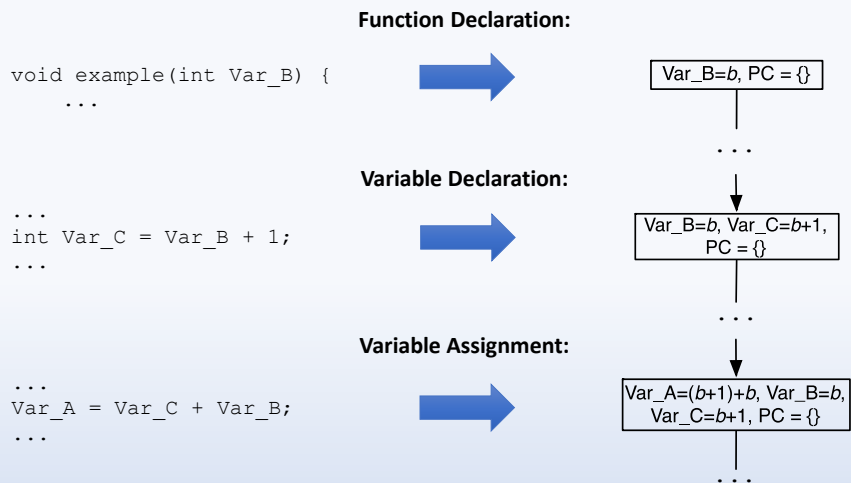
10

Symbolic Execution: Variables



11

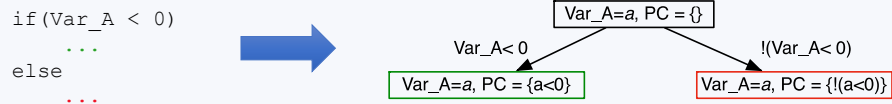
Symbolic Execution: Variables



12

Symbolic Execution: Conditionals

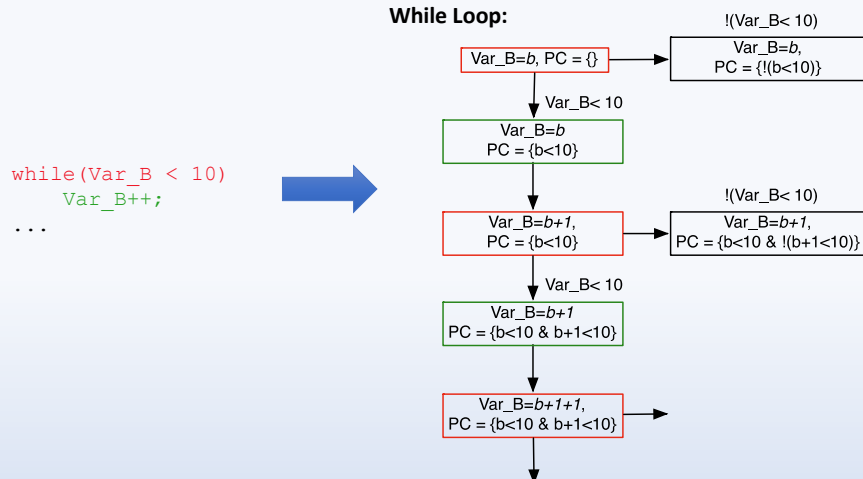
If Statement:



13

Symbolic Execution: Conditionals

While Loop:



14

Symbolic Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```

$A=a, B=b, PC=\{\}$

15

Symbolic Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```

$A=a, B=b, PC=\{\}$

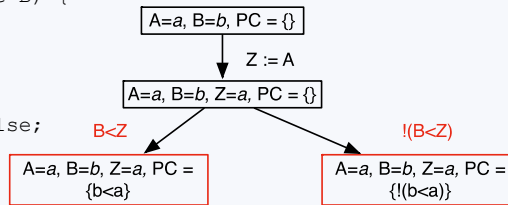
$Z := A$

$A=a, B=b, Z=a, PC=\{\}$

16

Symbolic Execution: Example

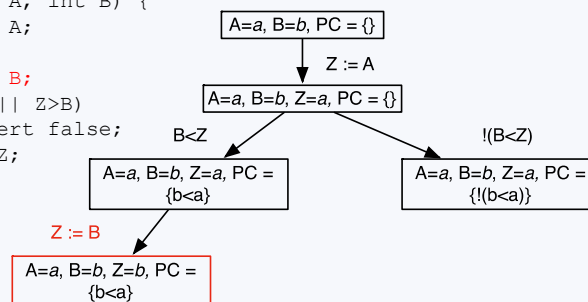
```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```



17

Symbolic Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```



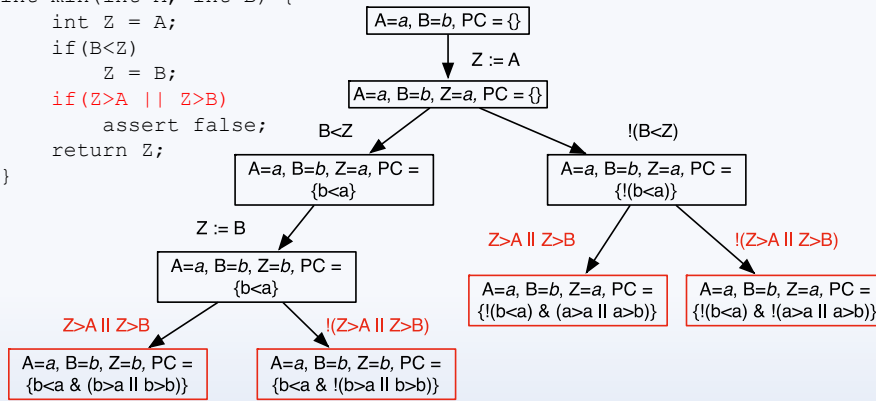
18

Symbolic Execution: Example

```

int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}

```



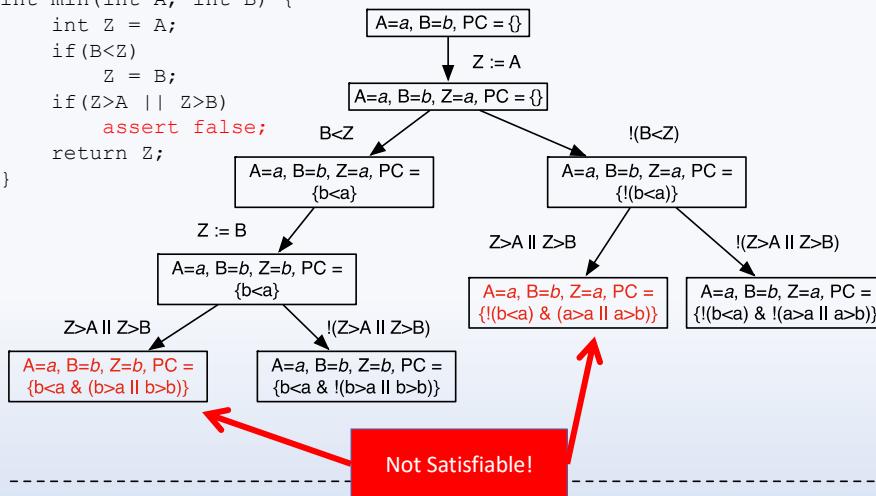
19

Symbolic Execution: Example

```

int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}

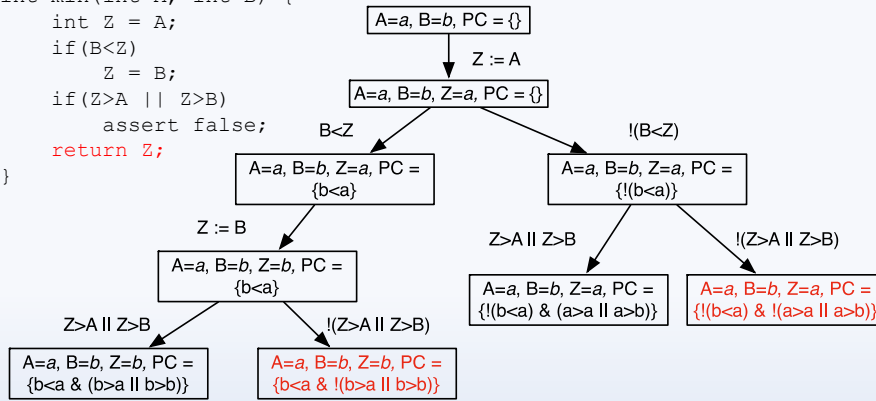
```



20

Symbolic Execution: Example

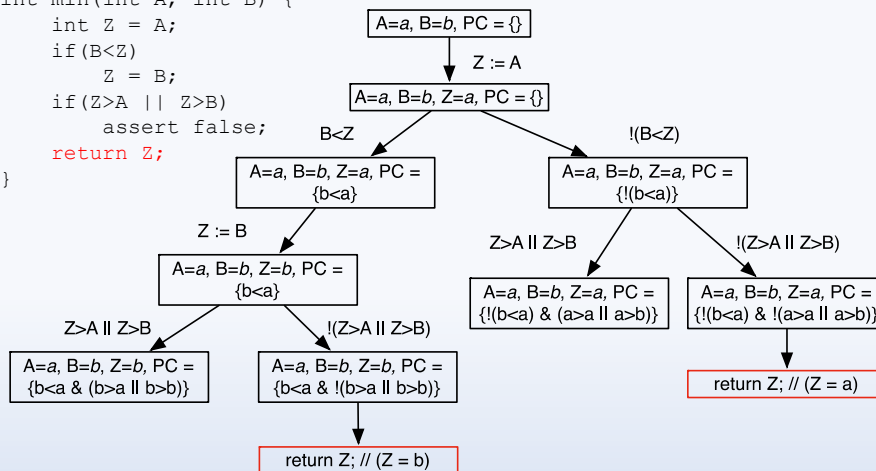
```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```



21

Symbolic Execution: Example

```
int min(int A, int B) {
  int Z = A;
  if (B < Z)
    Z = B;
  if (Z > A || Z > B)
    assert false;
  return Z;
}
```



22

How would you symbolically execute this?

```
bool Negative(int A) {  
    if(A < 0)  
        return true;  
    return false;  
}
```

23

How would you symbolically execute this?

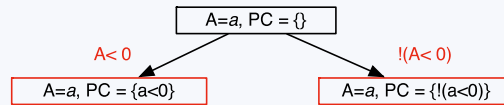
```
bool Negative(int A) {  
    if(A < 0)  
        return true;  
    return false;  
}
```

A=a, PC={}

24

How would you symbolically execute this?

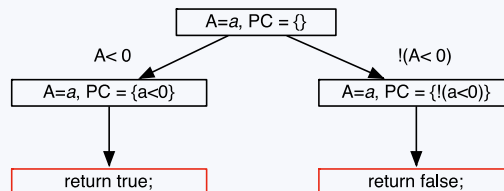
```
bool Negative(int A) {
  if(A < 0)
    return true;
  return false;
}
```



25

How would you symbolically execute this?

```
bool Negative(int A) {
  if(A < 0)
    return true;
  return false;
}
```



26

Limitations of Symbolic Execution

- **Path explosion** due to conditionals in loops
- **Solving path conditional**
- **Complex Inputs**
 - Arrays,
 - Data Structures (lists, trees, and so on)
- **Non-terminating programs**